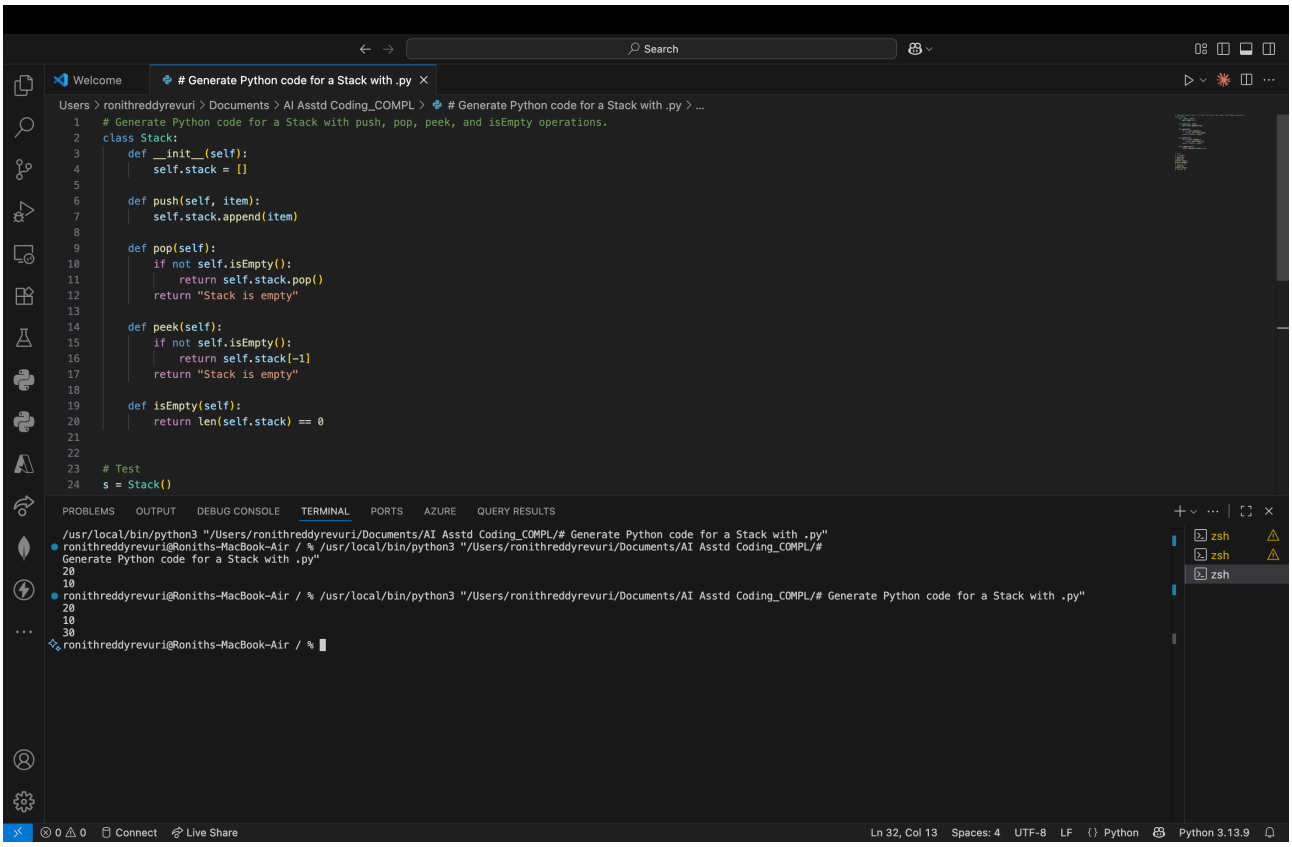
ASSIGNMENT-11.2

TASK DESCRIPTION -1 – (STACK USING AI GUIDANCE)

PROMPT :
GENERATE PYTHON CODE FOR A STACK WITH PUSH, POP, PEEK, AND ISEMPTY OPERATIONS.

EXPLANATION :
A STACK USES LIFO (LAST IN FIRST OUT). YOU ADD AND REMOVE ELEMENTS FROM THE TOP. IT IS HELPFUL FOR UNDO OPERATIONS AND RECURSION.

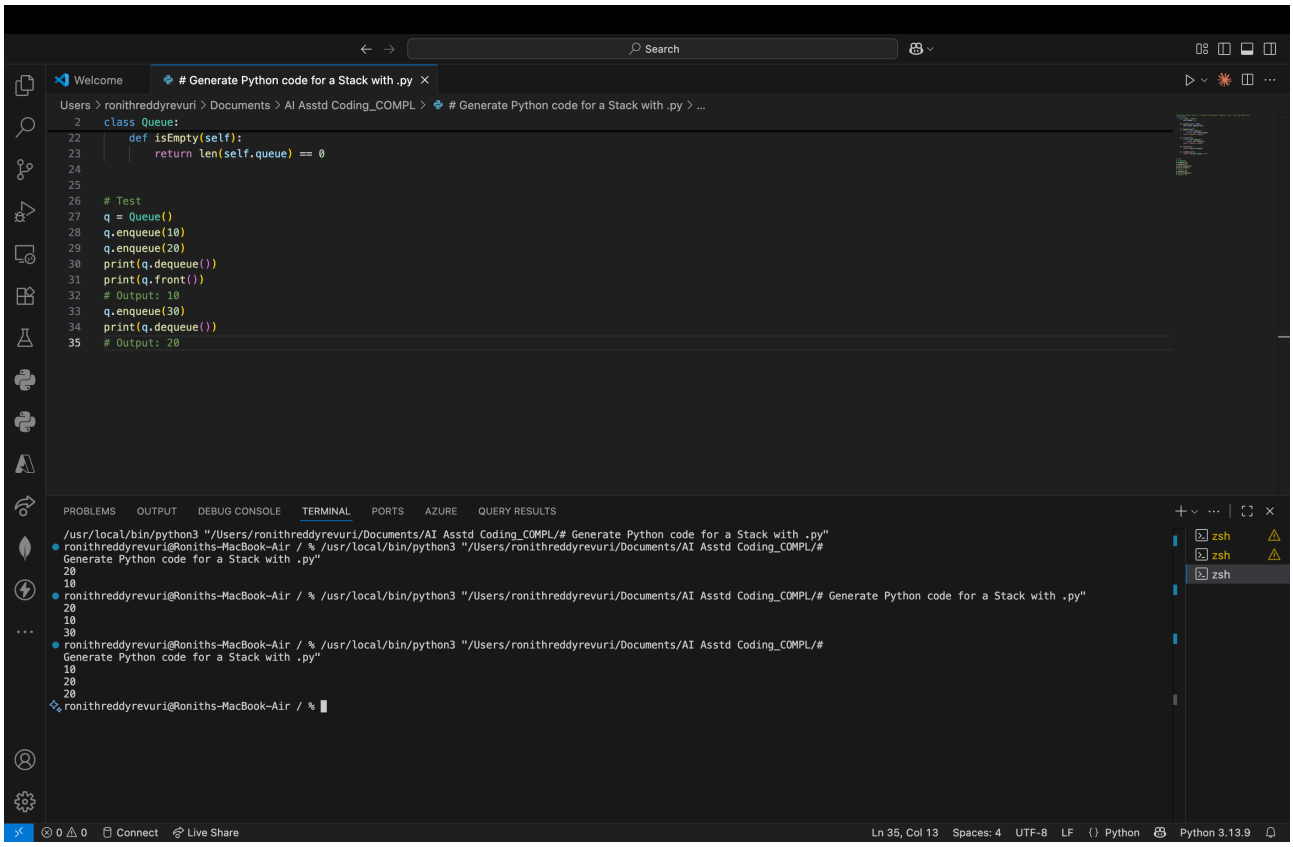


TASK 2 – QUEUE

PROMPT :
GENERATE PYTHON CODE FOR A QUEUE WITH ENQUEUE, DEQUEUE, FRONT, AND SIZE OPERATIONS.

EXPLANATION:

A QUEUE WORKS IN A SIMPLE WAY. IT FOLLOWS THE RULE THAT THE FIRST THING YOU ADD IS THE THING YOU TAKE OUT. SO WHEN YOU PUT THINGS IN A QUEUE THE FIRST THING YOU PUT IN IS THE THING THAT GETS TAKEN OUT. THIS IS WHAT WE CALL FIRST IN FIRST OUT. WE USE QUEUES FOR LOTS OF THINGS, LIKE SCHEDULING AND BUFFERING. THE QUEUE HELPS US KEEP THINGS IN ORDER.

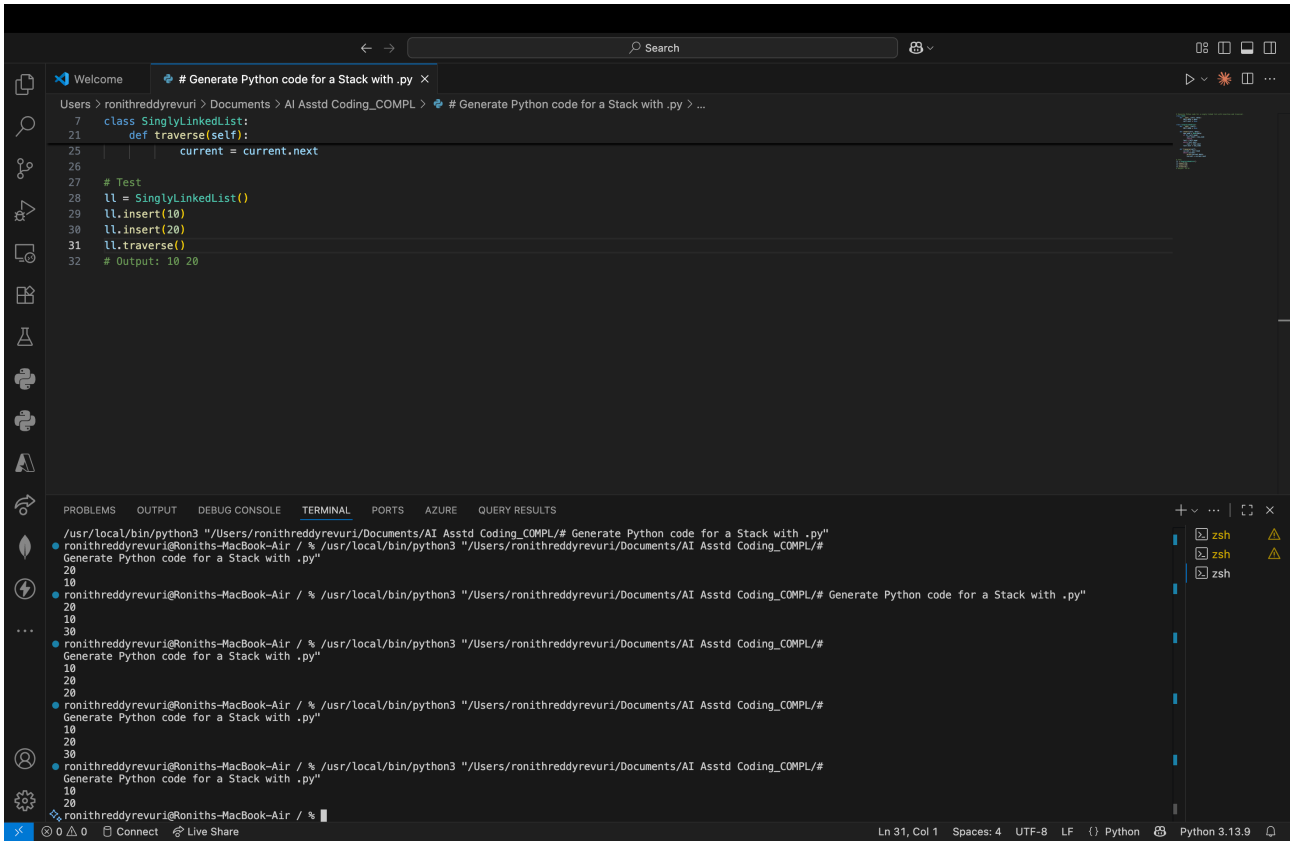


TASK 3 – SINGLY LINKED LIST

PROMPT :
GENERATE PYTHON CODE FOR A SINGLY LINKED LIST WITH INSERTION AND TRAVERSAL.

EXPLANATION :

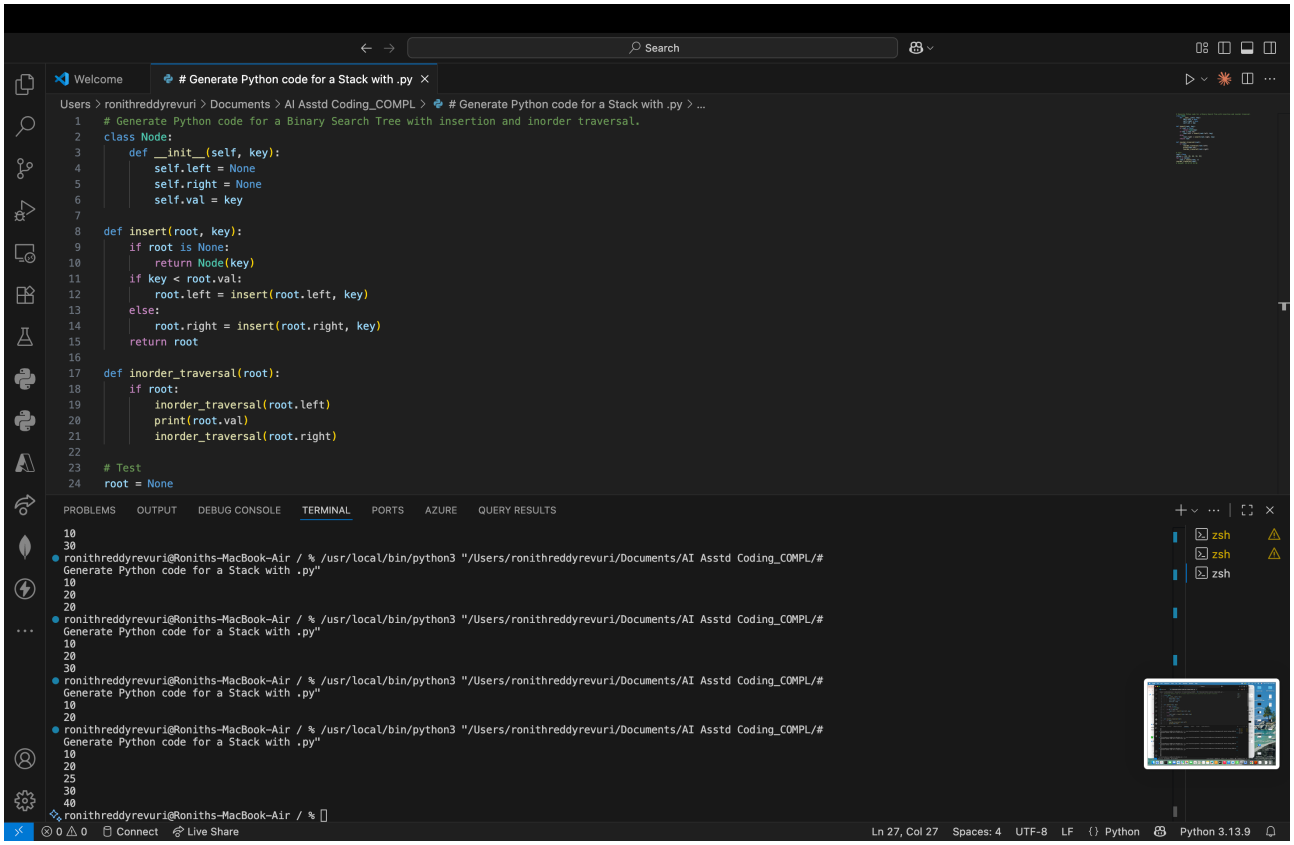
A LINKED LIST IS MADE UP OF ELEMENTS THAT ARE STORED IN THINGS CALLED NODES. THESE NODES ARE CONNECTED TO EACH OTHER BY POINTERS. THIS MEANS THAT A LINKED LIST CAN USE MEMORY IN A WAY THAT'S DYNAMIC. IT IS ALSO REALLY EASY TO ADD THINGS TO A LINKED LIST.



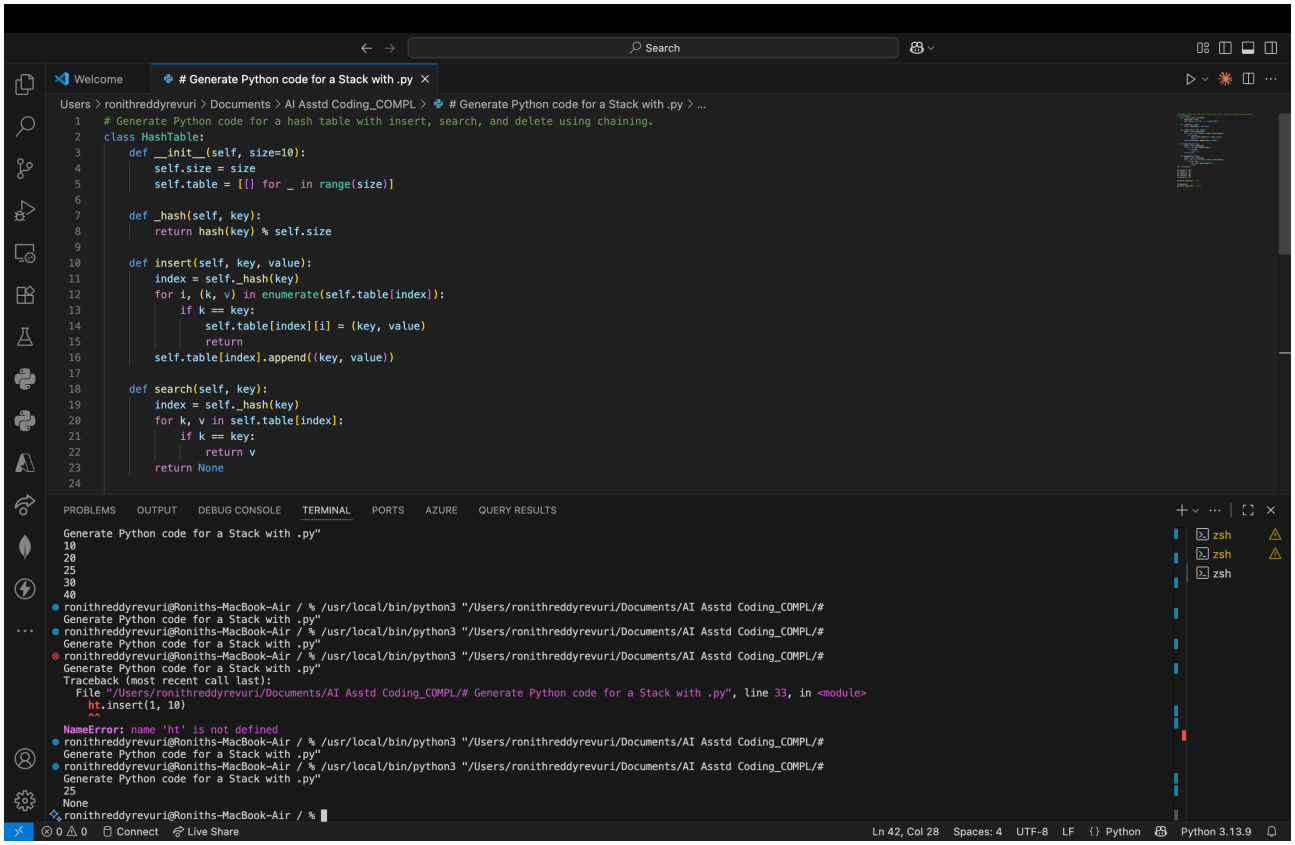
TASK 4 – BINARY SEARCH TREE

PROMPT :
GENERATE PYTHON CODE FOR A BINARY SEARCH TREE WITH INSERTION AND INORDER TRAVERSAL.

EXPLANATION :
A BINARY SEARCH TREE STORES DATA IN AN ORDER. THE LEFT CHILD OF A NODE IS ALWAYS SMALLER THAN THE NODE ITSELF AND THE RIGHT CHILD OF A NODE IS ALWAYS LARGER THAN THE NODE ITSELF. WHEN WE DO AN INORDER TRAVERSAL OF A BINARY SEARCH TREE IT PRINTS OUT THE VALUES IN AN ORDER. THIS IS BECAUSE THE INORDER TRAVERSAL OF A BINARY SEARCH TREE VISITS THE CHILD THEN THE NODE ITSELF AND FINALLY THE RIGHT CHILD. SO WHEN WE USE INORDER TRAVERSAL, ON A BINARY SEARCH TREE WE GET THE VALUES IN AN ORDER.



TASK 5 : HASH TABLE

PROMPT :
GENERATE PYTHON CODE FOR A HASH TABLE WITH INSERT, SEARCH, AND DELETE USING CHAINING.

EXPLANATION :
A HASH TABLE IS REALLY USEFUL FOR STORING DATA. IT USES A KIND OF FUNCTION CALLED A HASH FUNCTION. THIS HASH FUNCTION HELPS US FIND THE DATA WE NEED QUICKLY. WHEN THERE IS A PROBLEM, WITH TWO PIECES OF DATA TRYING TO BE STORED IN THE PLACE, WHICH IS CALLED A COLLISION THE HASH TABLE USES SOMETHING CALLED CHAINING TO DEAL WITH IT. THE HASH TABLE IS A WAY TO STORE DATA BECAUSE IT LETS US GET TO THE INFORMATION WE NEED SO FAST.